Learn Only What You Can Verify: Hardening the Label Channel of Online IoT DDoS Detectors

Shirish Giroti
Dept. of Computer Science and Engineering
IIITDM Kancheepuram
Roll No. CS23B2041
Email: cs23b2041@iiitdm.ac.in

***Abstract*—Online Internet of Things (IoT) Distributed Denial of Service (DDoS) detectors are often presented as robust to evasion because they keep adapting to streaming traffic. This paper challenges that assumption. We re-evaluate the Multi-Stage Adversarial Defense (MSAAD), a representative online framework, on IoTID20, CICIoT2023 and a time-ordered CIC-IDS2017 stream, using 15 random seeds and paired statistical tests. We find that MSAAD's resilience depends on an instant, correct label channel: evasion stays at or below 0.12% with immediate labels, but rises to 17–49% for valid-traffic transfer attacks when labels are removed, and a delay of 5,000 flows already lets 9–28% of attack traffic through. Label poisoning takes over the online learner, raising evasion to 99.7–99.96%, while MSAAD's own filter catches only 11–17% of valid adversarial flows on IoTID20. To close this gap, we propose verified-label learning with uncertainty-directed audits under a 10% budget. It keeps evasion at or below 1.2% under full label poisoning on all three datasets and cuts false alarms under attack by 8–13× compared with random audits. Finally, we show that trusting attack labels opens a framing attack. Under combined poisoning and framing, importance-weighted consensus updates with uncertainty-directed audits keep evasion at or below 0.65% when the attacker also perturbs its traffic, and at or below 1.7% when it does not, at the cost of 2–26% false alarms.**

***Index Terms*—Online intrusion detection, IoT DDoS detection, adversarial machine learning, data poisoning, label noise, active learning, concept drift.**

I. Introduction

The growth of IoT over the past decade has brought significant developments in modern digital infrastructure. IoT networks are the foundation for self-monitoring systems in manufacturing, automated crop telemetry, and smart city frameworks, speeding up technological progress by leaps and bounds. IoT has bridged the gap between the physical and digital worlds, bringing real-time data visibility, efficiency, and predictive analytics.

But the higher the mountain, the longer its shadow: the same connectivity that makes IoT useful also exposes it. IoT networks have many vulnerabilities waiting to be exploited by different types of attackers. For example, IoT botnets such as Mirai exploit vulnerable edge devices to orchestrate massive DDoS attacks [1] that overwhelm network resources.

While standard ML detectors achieve high accuracy in static, offline settings, real-world IoT traffic exhibits continuous concept drift, causing static models to lose accuracy over time. As a result, recent DDoS detection research is shifting towards online learning detectors that update in real time without full retraining [2], [3]. A frozen detector, once evaded, stays evaded; an online detector can recover, but only if the labels it learns from are correct.

As a representative baseline of recent online defenses, Beshah et al. [4] introduced the Multi-Stage Adversarial Defense (MSAAD) framework for online IoT DDoS detection, a three-layer architecture described in Section II. Evaluated on the IoTID20 and CICIoT2023 datasets, MSAAD reports a clean classification accuracy of 99.50% and 99.28%, respectively. While accuracy degrades to 38.92–67.12% on IoTID20 and 13.08–32.38% on CICIoT2023 under gradient-based evasion attacks (FGSM, BIM, PGD, C&W), the authors report that the RADP layer restores detection accuracy to roughly 99%.

Despite these figures, a methodological audit reveals three gaps that obscure real-world deployment performance:

1) *Instant ground-truth labels.* MSAAD's prequential protocol assumes that the true label y_i of every flow arrives instantly after prediction (Algorithm 1, line 13). Since the system gets instant and accurate labels, it is never tested against real-world issues like delayed or tampered labels. It only seems resilient because it quickly adapts to attack patterns after seeing a few flows.
2) *Unconstrained, all-flow perturbations.* These attacks modify scaled features without domain checks, so some resulting flows could not exist as real network traffic, such as negative values or a Min greater than the Mean. The method also perturbs benign flows, which artificially degrades precision. A real-world adversary only controls its own traffic.
3) *Missing security metrics.* MSAAD relies on overall accuracy, which is misleading under IoT class imbalance, where always predicting DDoS already yields high accuracy. It omits the false-positive rate (FPR), the operational burden on legitimate traffic, and the attack success rate (ASR), which measures evasion directly.

The gaps noticed in the existing defences give rise to a fundamental question: "What happens to an online detector when its labels are not instant and correct?" An online detector is only as robust as its label channel, meaning it should learn only what it can verify and direct verification precisely where it is unsure. We address this by treating the label channel

explicitly as an attack surface and evaluating MSAAD against late, missing, poisoned and framed label streams. We then propose verified-label learning, where our learner never learns an unverified benign label, re-weights audited benign flows, and accepts an unverified DDoS label only when the detector agrees, allocating a 10% audit budget to the flows with the highest prediction uncertainty. We evaluate this approach on IoTID20 [5], CICIoT2023 [6] and a time-ordered, corrected CIC-IDS2017 [7] using 15 seeds and Holm-corrected paired statistical tests. Our contributions are:

1) We define a threat model that incorporates the label channel and re-evaluate MSAAD, showing that evasion remains $\leq 0.12\%$ under instant labels, rises to 17–49% with no labels, and reaches 9–28% when labels are delayed by 5,000 flows.
2) Through a corrected evaluation that restricts attack perturbations to valid traffic and reports FPR and ASR, we show that label poisoning drives the learner to 99.7–99.96% evasion, while MSAAD's RADP filter detects only 11–17% of valid attacks on IoTID20.
3) We propose verified-label learning combined with uncertainty-directed audits, checked at matched detection rates, which restricts evasion to $\leq 1.2\%$ under full label poisoning while achieving an FPR under attack 8–13$\times$ lower than with random audits.
4) We identify framing as an availability attack created by blindly trusting DDoS labels, and show that under simultaneous poisoning and framing, evasion remains $\leq 1.7\%$ at a cost of 2–26% false alarms. We also document negative results for RADP, adversarial training (which halts transfer but doubles FPR), and a graph-based detection stage.

II. Background and Related Work

A. The Base System

Beshah et al. [4] recently introduced the Multi-Stage Adversarial Defense (MSAAD) framework for online DDoS detection in IoT networks. The architecture builds on their earlier drift-adaptive ensemble [2] by chaining three modules. First, a GAN-based Resilient Adversarial Detection and Purification (RADP) filter detects adversarial flows and deletes them before classification. Second, a pool of five drift-aware online learners classifies the remaining flows. Finally, a multi-armed bandit using Thompson sampling (MABTSE) selects the top three learners for each flow and combines their outputs with an accuracy-update weighted probability averaging ensemble (AUWPAE). The authors report post-defence accuracies of roughly 99% on public benchmarks, but their evaluation rests on two unstated choices that we examine in Section III: ground-truth labels arrive immediately, and the attacker perturbs every flow without domain constraints.

B. Online Detection and Concept Drift

Online network intrusion detection needs streaming algorithms that process continuous traffic within tight memory limits. Adaptive classifiers such as Adaptive Random Forests (ARF) [8] and Streaming Random Patches (SRP) [9] handle drift by embedding explicit drift detectors such as ADWIN [10] or DDM [11]. Performance-weighted ensembles such as PWPAE [3] combine such learners, and libraries such as River [12] implement them. Unsupervised pipelines such as Kitsune [13] avoid labels by flagging anomalies, and CADE [14] detects and explains drifting samples in security data. Delayed-label protocols [15], [16] model realistic feedback lag, but they assume the late feedback is still correct. Our work treats the label feedback loop itself as an attack surface exposed to delay, poisoning and framing.

C. Realistic Evasion of Network Detectors

Evasion attacks commonly use gradient methods such as FGSM [17], BIM [18] and PGD [19], optimisation-based attacks such as C&W [20], or decision-based queries such as HopSkipJump [21]. Applied to network intrusion detection, feature perturbations must respect domain constraints; otherwise, the resulting traffic could not exist [22]. Realistic adversaries can also manipulate only their own traffic [23], and methodology guidelines warn against judging detectors by overall accuracy under severe class imbalance [24]. Our evaluation therefore perturbs attack flows only, repairs every adversarial flow to valid traffic, and reports the attack success rate (ASR) and false-positive rate (FPR).

D. Poisoning, Label Noise and Trusted Labels

Data poisoning attacks corrupt a model by manipulating its training data, from crafted points against support vector machines [25] to attacks on online learners as they update [26]. In network security, attackers can plant clean-label backdoors in batch-trained flow classifiers [27]. Older attacks on signature learners mislead the learner with crafted samples (Paragraph [28]) or trick it into signatures that block legitimate traffic (Allergy [29]); the latter is a precursor of the framing attack we study. Defences against label noise include Gold Loss Correction (GLC), which uses a small trusted set [30], and active learning [31], which spends a labelling budget on informative samples; importance-weighted active learning (IWAL) keeps such sampling unbiased [32], and Lin et al. study active learning under malicious mislabelling on image data [33]. Our strategy brings these ideas into an online NIDS: we combine trusted audits, fixed-count duplication of audited benign flows, and uncertainty-directed audits to limit label delay, poisoning and framing together.

E. Defences and Adaptive Attacks

Detect-and-reject defences often collapse against adaptive attackers [34] or rely on obfuscated gradients [35], so evaluation guidelines call for testing each defence against attacks tailored to it [36]; adversarial training [19] remains the standard baseline. We test the RADP filter against domain-constrained attacks it was not tuned for, and compare adversarial training against label-only HopSkipJump attacks.

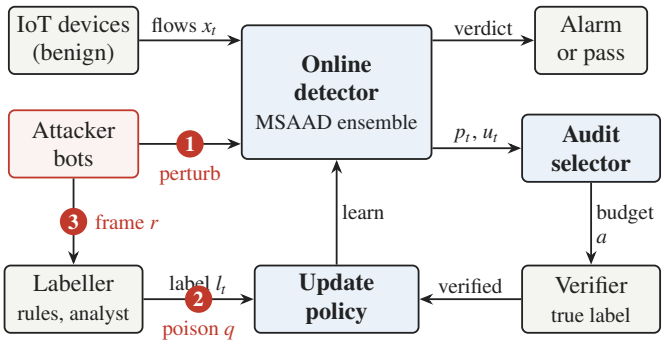


Fig. 1. System overview. The online detector classifies each flow x_t; the audit selector uses the uncertainty u_t and the budget a to send some flows to the verifier, and the update policy decides which labels the detector learns from. Numbered circles mark the attacker's three entry points: (1) perturbation of its own flows, (2) label poisoning with rate q, and (3) framing with rate r.

F. Graph and Temporal Detectors

Graph-based intrusion detectors build relational structures over flow endpoints, using graph neural networks such as E-GraphSAGE [37], temporal graph models in which simple MLP-based designs can match complex ones [38], or streaming edge counters such as MIDAS [39]. In our detector chain, a host-graph GNN does not outperform a parameter-matched MLP.

We are not aware of prior work that evaluates an online IoT DDoS detector's label channel under simultaneous delay, poisoning and framing, or that defends it with verified-label learning.

III. System and Threat Model

A. The Online Detector and Its Label Channel

Our online detector runs a prequential (test-then-train) streaming loop (Fig. 1): it classifies each incoming flow x_t and then learns from a label l_t supplied by a labeller. In real deployments, these labels come from signature rules, analyst triage, threat-intelligence feeds, or the model's own earlier predictions. Rule latency, stale feeds and low-rate evasion mean that labels are often delayed, missing, or wrong. To verify uncertain traffic, the defender can spend an audit budget of $a = 10\%$ of flows on forensic checks or an expert verifier, which returns the true label y_t. MSAAD uses a vanilla update policy that trains on every incoming label immediately, without verification (Algorithm 1, line 13); our approach instead filters the labels through a verified-label update policy.

B. Attacker Goals, Knowledge and Capabilities

We model a grey-box adversary with two goals: evasion (G1), getting DDoS flows past the detector, measured by the attack success rate (ASR); and availability disruption through framing (G2), getting benign flows flagged as DDoS, measured by the false-positive rate (FPR) on genuine benign flows. The attacker knows the feature space and trains a local surrogate on similar IoT traffic, but cannot access the detector's weights or its audit selection. It has three capabilities. First, it can perturb its own attack flows within an L_∞ bound of $\epsilon = 0.1$ on scaled features, provided the result remains valid traffic. Second, it can cause the labeller to mark each of its flows benign with probability q (label poisoning). Third, it can frame benign traffic: its hosts send benign-looking flows at a rate of r per test flow, which the labeller correctly marks as DDoS because they come from attacker hosts. Independently of the attacker, labels may also arrive late by a fixed number of flows or seconds.

In a live deployment, an attacker raises q by exploiting blind spots in rule-based and self-labelling systems. Low-rate or pulsed floods stay below volume thresholds, avoiding alerts and getting labelled benign by default [26]. Attackers can also rotate or spoof source IPs to bypass standard threat blocklists. In self-labelling systems the problem escalates, as every successful evasion gets learned as a benign training sample. This is made worse by the fact that security data streams often have late or noisy labels anyway [15], [16]. While $q = 1$ is the worst case, real levels depend on the labelling infrastructure, so we test $q \in \{0, 0.1, 0.25, 0.5, 1\}$.

C. Out of Scope

We do not consider direct compromise of the audit verifier, attacks on the feature extractor, or packet-level replay. A more adaptive adversary could also shape its poisoned or framing flows to avoid uncertainty-directed audits; attacks that steer uncertainty-based selection exist in other domains [40]. We treat this as an open threat and return to it among the limitations in Section VI.

IV. Methodology

A. Datasets and Stream Protocol

We evaluated our framework on three network intrusion datasets, summarized in Table I: IoTID20 [5], CICIoT2023 [6], and the corrected version of CIC-IDS2017 [7], [41]. From IoTID20 we used normal traffic and the Mirai UDP, HTTP and ACK floods; we excluded brute-force flows because they are not DDoS. This data keeps its published file order and has 79 features. From CICIoT2023 we used one CSV part with benign traffic and all DDoS classes, 46 features, in file order. Because neither benchmark is truly time-ordered, we also used the Friday capture of the corrected CIC-IDS2017 (20:31–21:26 on the file clock) as a strictly time-ordered stream.

Each run used 30,000 evenly spaced flows. The first 20,100 (67%) trained the detector, and the remaining 9,900 formed the prequential (predict-then-learn) test stream. DDoS flows make up 87.6%, 96.6% and 75.4% of the test streams of IoTID20, CICIoT2023 and CIC-IDS2017. To prevent leakage, a running min-max scaler transforms each flow before the model learns from it, and IP addresses and timestamps are never used as features.

Two dataset caveats shape these experiments. First, IoTID20's file order is only about 50% time-ordered, and all its benign flows come from the first capture week, so any drift it shows is an artefact rather than a real traffic shift. Second, CICIoT2023 has only about 330 benign test flows, so a single

TABLE I
THE THREE STREAMS AND HOW THEY WERE BUILT. FEAT.: NUMBER OF FEATURES; ORDER: FILE ORDER OR TIME ORDER; DDOS (TEST): SHARE OF DDOS FLOWS IN THE 9,900-FLOW TEST STREAM.

Dataset	Classes used	Feat.	Order	DDoS (test)
IoTID20	Normal; Mirai UDP, HTTP and ACK floods	79	file	87.6%
CICIoT2023	Benign; DDoS-* (one CSV part)	46	file	96.6%
CIC-IDS2017 (corrected)	Friday BENIGN and DDoS, 20:31–21:26	79	time	75.4%

misclassified flow moves the false-positive rate (FPR) by about 0.3 points.

B. Detector Reimplementation

Because no source code was released, we reimplemented layers 2 and 3 of MSAAD from scratch with the River 0.22 library [12]. The classifier pool has five online learners: Adaptive Random Forest with ADWIN (ARF-ADWIN), ARF with DDM (ARF-DDM), Streaming Random Patches with ADWIN (SRP-ADWIN), SRP with DDM (SRP-DDM), and k-nearest neighbours with ADWIN (KNN-ADWIN). Following the base paper, the ARF models use 3 trees, the SRP models 3 base learners, and KNN-ADWIN uses $k = 5$ over a window of 100 flows.

A Thompson-sampling bandit [42] with Beta(1,1) priors selects the top three learners for each flow, and AUWPAE combines their outputs with weights $1/(\mathrm{MSE}_j + 10^{-4})$, where MSE_j is learner j's error over the last 500 flows. The base paper leaves two details open, so we made two assumptions: the bandit's reward is each learner's correctness on every flow, and KNN-ADWIN was rebuilt to fit River 0.22.

C. Attacks and Feasibility Repair

For the grey-box threat, we trained an MLP surrogate on the training split. It generated FGSM, BIM, PGD and C&W attacks with $\epsilon = 0.1$ on scaled features. We only perturbed DDoS flows, unlike MSAAD, which changes benign traffic too. To fix impossible features, we mined rules from clean dataset flows (constant, binary, categorical, integer and non-negative features, equal groups, and $\mathrm{Min} \le \mathrm{Mean} \le \mathrm{Max}$). The repair step pushes each adversarial flow back inside these rules without increasing any single feature's perturbation. We report both raw and repaired results. Finally, we tested decision-based adaptive attackers using HopSkipJump [21] with 2,000 queries per flow.

D. Label Feedback Modes

We evaluated five feedback modes. *Instant* feedback follows the base paper: the true label arrives right after each prediction (Algorithm 1, line 13). *None* (frozen) means the detector receives no updates during the test stream. *Flow-delayed* feedback holds back each label by 1,000 or 5,000 flows. *Time-delayed* feedback holds labels back by 60 or 300 seconds; it runs only on the timestamped CIC-IDS2017 stream. Finally, with *self-labelling* the detector learns from its own earlier verdicts.

Algorithm 1 Prequential test loop with audits and update policy π. $\mathcal{S}$ is the set of policies that never learn an unverified benign label.

Require: detector f, policy π, audit budget a, copies $c = \min(\mathrm{round}(1/a), 20)$; $\mathcal{S} = \{$strict, IW-strict, consensus, IW-consensus$\}$

1: **for** each test flow x_t with labeller label l_t **do**
2: $p_t \leftarrow f(x_t)$; predict $\hat{y}_t = \arg\max$
3: $d_t \leftarrow \textsc{Random}(a)$ or $\textsc{Uncertain}(u_t, a)$
4: **if** d_t **then**
5: $y_t \leftarrow$ verified label; learn (x_t, y_t)
6: **if** $y_t =$ benign and π is IW-strict or IW-consensus **then**
7: learn (x_t, y_t) another $c - 1$ times
8: **end if**
9: **else if** $l_t =$ benign and $\pi \in \mathcal{S}$ **then**
10: skip {unverified benign label}
11: **else if** $l_t =$ DDoS, $\hat{y}_t =$ benign, $\pi \in \{$consensus, IW-consensus$\}$ **then**
12: skip {needs the detector's agreement}
13: **else**
14: learn (x_t, l_t)
15: **end if**
16: **end for**

E. Update Policies

We compared several update policies, formalized in Algorithm 1. *Vanilla* updates the model on every incoming label without checking it, as MSAAD does. *Audit* uses verified labels only to update the Thompson-sampling selector and leaves the learners unchanged. *Guard* checks suspicious benign labels before allowing an update, while *Strict* never learns an unverified benign label: only audited benign flows are learned.

IW-strict also duplicates each audited benign flow $c = \min(\mathrm{round}(1/a), 20)$ times, which gives $c = 10$ copies at our 10% audit budget. It is inspired by importance-weighted active learning [32], but uses fixed copies instead of inverse-probability weights. *Consensus* and *IW-consensus* add one rule: an unverified DDoS label is learned only if the ensemble's own prediction agrees with it. Finally, the *noise-aware* baseline estimates the poisoning rate $\hat{\rho}$ from audit outcomes and learns unverified benign labels with probability $1 - \hat{\rho}$.

To simulate label poisoning, we relabelled each attacker flow as benign with probability $q \in \{0, 0.1, 0.25, 0.5, 1\}$. For framing, we replayed benign training flows from attacker hosts, which the labeller marks as DDoS, at rates $r = 2\%$ or $r = 10\%$ per test flow during the first half of the test stream. These framing flows are excluded from every metric, so the metrics measure genuine traffic only. We also evaluated a combined attack: evasion with repaired PGD, full poisoning ($q = 1$) and framing at $r = 5\%$.

F. Audit Allocation

With a budget of $a = 10\%$, we tested three allocation strategies. *Random* allocation audits each flow with probability a.

Uncertainty-directed allocation audits the flows the ensemble is least sure about [31]. *Mixed* allocation splits the budget equally between the two. For flow t, the uncertainty is

$$u_t = 1 - \big|2\,p_t(\mathrm{DDoS}) - 1\big|, \tag{1}$$

where p_t is the ensemble's probability that the flow is DDoS, so $u_t = 0$ when the model is certain and $u_t = 1$ when $p_t = 0.5$. A flow is audited only if u_t is at or above the $(1 - a)$ quantile of the previous 500 values and the audit credit, which grows by a per flow, is at least 1. This credit rule ensures that audits never exceed the budget.

G. Baselines

We compared against three baselines. First, we rebuilt RADP (layer 1 of MSAAD): MSAPG perturbation generation, GAN sampling, the RAD detector and delete-on-flag filtering, plus a leave-one-attack-out variant, over 5 seeds. Second, for adversarial training [19] we added one repaired PGD copy of each DDoS training flow, crafted on the defender's own surrogate. Third, a detector chain places an online MLP stage after the ensemble, followed by a host-graph attention network, which we compared with a parameter-matched host-feature MLP; the chain's stage thresholds were calibrated to 1% FPR.

H. Metrics and Statistical Analysis

We measured the attack success rate (ASR) over genuine attack flows A, and the FPR over genuine benign flows B:

$$\mathrm{ASR} = \frac{100}{|A|}\,\big|\{x \in A : \hat{y}(x) = \mathrm{benign}\}\big|, \tag{2}$$

$$\mathrm{FPR} = \frac{100}{|B|}\,\big|\{x \in B : \hat{y}(x) = \mathrm{DDoS}\}\big|. \tag{3}$$

We also report the FPR during and after framing campaigns, the number of audits used, the ROC AUC, the FPR at 99.9% and 99% detection, and the detection rate at 1% FPR, each with one threshold per run set after the run.

We ran every configuration with 15 seeds, and 5 seeds for the more expensive RADP and HopSkipJump experiments. Seeds control model initialization, surrogate training, attack perturbations and audit draws; they do not change the data sample. We used paired two-sided Wilcoxon signed-rank tests with Pratt zero-handling [43] and Holm correction within each comparison family [44], and we report 95% bootstrap confidence intervals (10,000 resamples) and rank-biserial effect sizes. With 15 seeds, the smallest attainable p-value is 6.1×10^{-5}.

All runs are deterministic: a configuration repeated in another grid reproduces bit for bit. Everything ran on a CPU-only machine (Intel i7-14650HX, 16 GB RAM); the full grid took about 40 hours.

V. Results

Triples report IoTID20 / CICIoT2023 / CIC-IDS2017, and values are means over 15 seeds unless stated otherwise.

TABLE II
REPRODUCTION AGAINST THE BASE PAPER: ACCURACY AND FPR ON CLEAN TRAFFIC, % (MEAN ± STD). PAPER: ACCURACY REPORTED IN [4]; –: NOT REPORTED. THE LAST COLUMN IS THE ACCURACY OF ALWAYS ANSWERING DDOS.

	Paper	Ours, 50k	Ours, 30k flows, 15 seeds		
Dataset	acc.	acc., 3 seeds	acc.	FPR	all-DDoS
IoTID20	99.50	99.33±0.08	99.54±0.08	3.33±0.63	87.6
CICIoT2023	99.28	99.76±0.10	99.71±0.09	5.77±2.24	96.6
CIC-IDS2017	–	–	99.93±0.02	0.12±0.08	75.4

A. Reproduction and False Alarms

Our reimplementation reproduces the reported clean accuracy, but accuracy hides the false alarms. On a 50,000-flow stream over 3 seeds (Table II), it reaches 99.33% on IoTID20 ($F_1 = 99.62$) against the reported 99.50% ($F_1 = 99.74$), and 99.76% on CICIoT2023 against the reported 99.28%. Under our main 30,000-flow protocol, accuracy is 99.54 / 99.71 / 99.93%. The base paper does not report the FPR, which is 3.33 / 5.77 / 0.12% with MSAAD's own learning rule. Because DDoS dominates these streams, always answering DDoS already scores 87.6 / 96.6 / 75.4% accuracy, so accuracy alone says little about the cost to analysts.

B. Evasion and the Label Channel

MSAAD's resilience comes from instant, correct labels rather than from the model. With instant labels (Table III, Fig. 2), repaired PGD reaches an ASR of only 0.10 / 0.08 / 0.02%, and every attack stays at or below 0.12%. With no labels, repaired PGD evasion rises to 17.32 / 48.52 / 45.36% (medians 17.0 / 73.6 / 39.3%; $p_{\mathrm{Holm}} \le 0.0014$), and every seed is worse than with instant labels on IoTID20 and CICIoT2023, as are 14 of 15 on CIC-IDS2017.

Delayed labels fall in between: for PGD, a 1,000-flow delay gives 1.80 / 5.27 / 6.03%, and a 5,000-flow delay gives 8.65 / 26.17 / 28.23%. On the time-ordered CIC-IDS2017 stream, delays of 60 and 300 s give 6.96% and 35.75%. Self-labelling does not protect the detector either (15.03% on IoTID20 and 66.70% on CICIoT2023, 3 seeds). Raw and repaired attacks transfer equally well ($p_{\mathrm{Holm}} \ge 0.46$), so the evasion lives in features an attacker can really change.

C. Label Poisoning and Verified-Label Learning

Verified-label learning stops poisoning, and importance weighting reduces its false-alarm cost. Under evasion with poisoning (Fig. 3, Table IV), vanilla evasion grows with $q = 0.25/0.5/1$ from 3.37 to 51.50 and 99.74% on IoTID20, and from 1.60 to 50.55 and 99.96% on CICIoT2023; on CIC-IDS2017 it reaches 99.95% at $q = 1$. The failure switches on between $q = 0.25$ and $q = 0.5$. Updating only the bandit selector (audit policy) does not help (99.59 / 99.84 / 99.82%), and guard is unstable at $q = 1$ (83.73 ± 23.97, 62.18 ± 45.54, $65.98 \pm 48.24\%$).

With a 10% random audit budget at $q = 1$, strict (0.88 / 0.44 / 0.14%) and IW-strict (1.15 / 0.43 / 0.13%) both keep

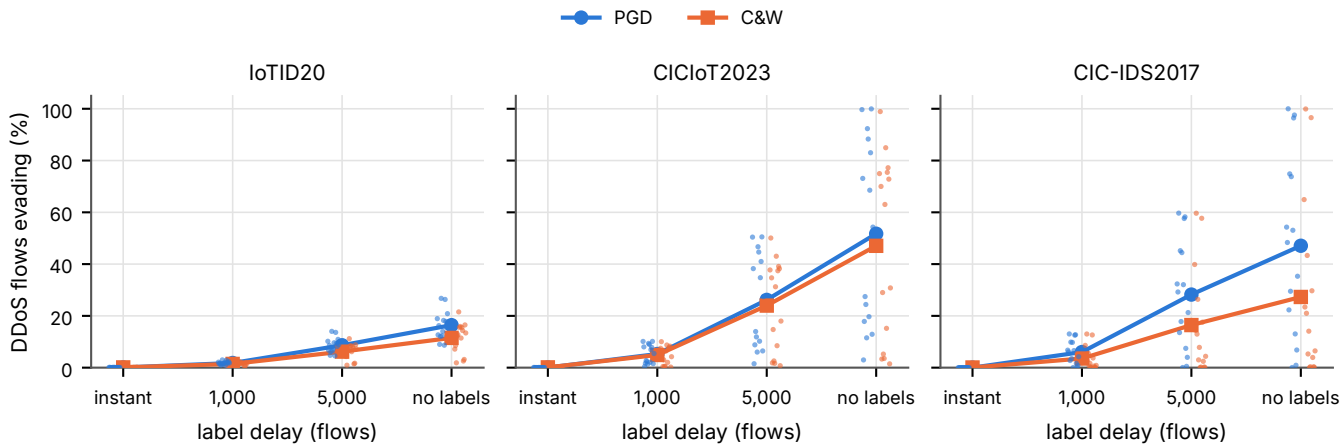


Fig. 2. Evasion (ASR) of PGD and C&W transfer attacks as the label delay grows, from instant labels to no labels, for each dataset. Each dot is one seed.

TABLE III
EVASION (ASR, %) BY LABEL MODE, MEAN ± STD OVER 15 SEEDS. †3 SEEDS. REPAIRED: ADVERSARIAL FLOWS PROJECTED BACK INTO THE VALID FEATURE SPACE (SECTION IV-C). THE MEDIAN ROW GIVES THE MEDIANS OF THE ROW ABOVE IT. –: NOT RUN; TIME DELAYS NEED TIMESTAMPS, WHICH ONLY CIC-IDS2017 HAS.

Label mode, attack	IoTID20	CICIoT2023	CIC-IDS2017
Instant (Alg. 1), PGD	0.07±0.03	0.04±0.02	0.01±0.01
Instant (Alg. 1), repaired PGD	0.10±0.03	0.08±0.03	0.02±0.01
Delay 1,000 flows, PGD	1.80±0.61	5.27±3.64	6.03±4.65
Delay 5,000 flows, PGD	8.65±2.80	26.17±18.32	28.23±21.20
Delay 60 s, PGD	–	–	6.96±5.33
Delay 300 s, PGD	–	–	35.75±26.87
No labels, PGD	16.42±5.41	51.73±36.20	47.09±35.40
No labels, repaired PGD	17.32±4.22	48.52±39.02	45.36±34.10
median	17.00	73.58	39.33
No labels, repaired C&W	11.25±6.03	47.69±37.38	25.21±33.93
Self-labels, PGD†	15.03±1.13	66.70±57.67	–

evasion low ($p_{\text{Holm}} \leq 0.007$ against vanilla). The cost appears in the FPR under attack with true labels: strict raises it to 5.33 / 23.12 / 0.65%, against vanilla's 2.17 / 3.52 / 0.11%, while IW-strict lowers it to 3.52 / 6.95 / 0.25%, below strict on all three datasets ($p_{\text{Holm}} \leq 0.0004$). A 20% budget narrows the gap further (2.64 / 4.68 / 0.21%).

D. Uncertainty-Directed Audits

Directing audits at uncertain flows lowers false alarms under attack while using fewer audits than random allocation (Table V): 899–911 / 897–902 / 727–818 audits, against 992 for random audits. With true labels under attack, IW-strict's FPR falls from 3.52 to 0.45%, from 6.95 to 0.64%, and from 0.25 to 0.02% ($p_{\text{Holm}} \leq 0.005$), an 8–13× reduction that also puts it below vanilla on all three datasets ($p_{\text{Holm}} \leq 0.006$). Under full poisoning, evasion falls only on IoTID20 (strict $0.88 \to 0.28\%$, IW-strict $1.15 \to 0.31\%$, $p_{\text{Holm}} \leq 0.004$); the other datasets show no significant change.

At a matched operating point that catches 99.9% of attacks, uncertainty audits cut IW-strict's FPR under attack from 3.92 to 0.71% on IoTID20 and from 5.87 to 0.64% on CICIoT2023 ($p_{\text{Holm}} \leq 0.012$), and the ROC AUC rises. On clean IoTID20 traffic the effect reverses: the FPR rises from 3.46 to 10.39% (vanilla 2.90%) and the AUC falls from 99.84 to 99.75%; mixed audits do not remove this (10.64%).

E. Framing

Trusting attack labels exposes strict policies to framing (Fig. 4). With random audits and $r = 2\%$, strict flags 61.3 / 92.3 / 22.6% of genuine benign flows, against 11.1 / 25.5 / 2.5% for vanilla. Uncertainty-directed audits lower the FPR in 34 of 36 whole-stream comparisons and in all 36 after the campaign ($p_{\text{Holm}} \leq 0.012$). After a 10% campaign, IW-strict's FPR falls from 36.2 to 8.9%, 43.0 to 1.6%, and 2.5 to 0.7%, below vanilla's 14.7 / 15.6 / 0.9%. IW-consensus with uncertainty audits is never significantly worse than vanilla, during or after the campaign. CIC-IDS2017 is far less sensitive to framing: vanilla reaches 7.1% at $r = 10\%$, against 37–51% on the other two datasets. Strict update rules should therefore not be combined with trusted attack labels without a consensus check.

F. The Combined Attack

IW-consensus with uncertainty-directed audits gives the best trade-off under the combined attack, at a measurable cost (Table VI). With evasion (repaired PGD), full poisoning ($q = 1$) and framing ($r = 5\%$), it keeps the ASR at 0.65 / 0.35 / 0.15%, while vanilla reaches 99.74 / 99.93 / 99.95%, and its FPR of 5.23 / 13.75 / 1.95% is below vanilla's 8.75 / 29.83 / 3.51% ($p_{\text{Holm}} \leq 0.006$). Against IW-strict, it gives lower false alarms on all three datasets ($p_{\text{Holm}} \leq 0.018$) with no significant difference in evasion, except on CICIoT2023 (0.35 vs 0.21%).

Without the evasion step (poisoning and framing only), IW-consensus holds the ASR to 1.67 / 0.24 / 0.14%, but its FPR of 11.27 / 26.07 / 2.65% is higher than vanilla's 7.16% on IoTID20 (Holm-adjusted $p = 0.004$), not significantly different on CICIoT2023 (25.01%), and lower on CIC-IDS2017

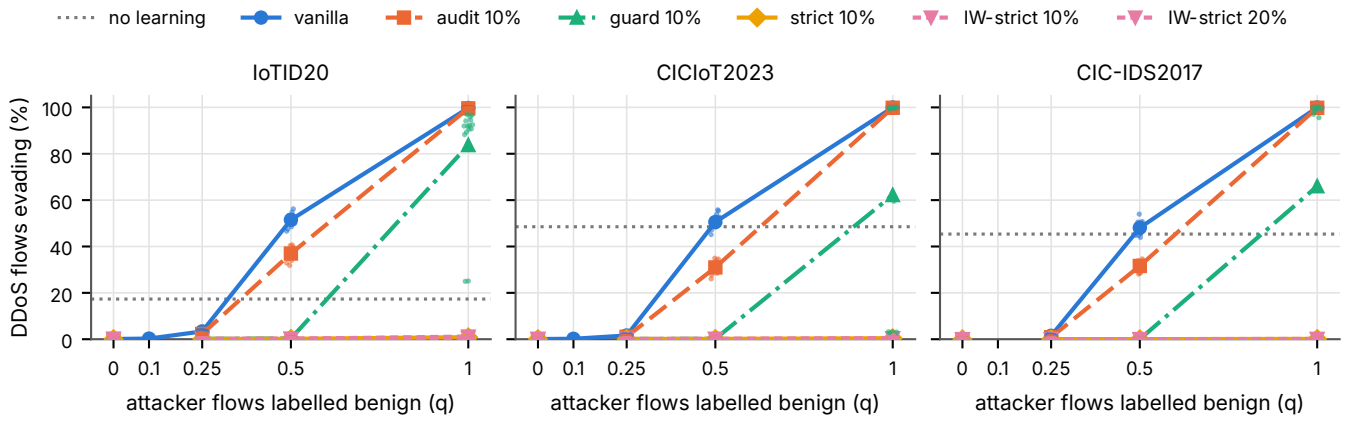


Fig. 3. Evasion under evasion plus label poisoning as the share q of attacker flows labelled benign grows, for each update policy (10% random audits unless stated). Dotted line: the detector without learning. Each dot is one seed.

TABLE IV

UPDATE POLICIES UNDER FULL POISONING AND THEIR COST: ASR UNDER EVASION PLUS POISONING ($q = 1$); FPR WITH TRUE LABELS ON CLEAN TRAFFIC AND UNDER ATTACK. 10% RANDOM AUDITS UNLESS STATED; %, MEAN ± STD OVER 15 SEEDS. ‡IDENTICAL TO IW-STRICT: WITH $q = 1$ NO UNVERIFIED DDOS LABEL REACHES THE LEARNER, SO THE CONSENSUS CHECK NEVER APPLIES.

	IoTID20			CICIoT2023			CIC-IDS2017		
Policy	ASR	FPR clean	FPR attack	ASR	FPR clean	FPR attack	ASR	FPR clean	FPR attack
No labels (frozen)	17.32±4.22	3.28±0.70	3.28±0.70	48.52±39.02	6.41±2.55	6.41±2.55	45.36±34.10	0.21±0.20	0.21±0.20
Vanilla (base paper)	99.74±0.08	3.33±0.63	2.17±0.69	99.96±0.03	5.77±2.24	3.52±2.01	99.95±0.06	0.12±0.08	0.11±0.05
Audit (selector only)	99.59±0.09	3.31±0.65	2.28±0.72	99.84±0.11	6.27±2.93	3.88±2.31	99.82±0.10	0.12±0.10	0.13±0.06
Guard	83.73±23.97	3.75±0.77	2.41±0.91	62.18±45.54	6.91±3.56	6.15±4.37	65.98±48.24	0.20±0.18	0.16±0.11
Strict	0.88±0.29	4.53±1.06	5.33±1.78	0.44±0.22	12.61±4.12	23.12±15.26	0.14±0.15	0.39±0.27	0.65±0.30
IW-strict	1.15±0.28	4.21±0.89	3.52±0.74	0.43±0.23	10.25±4.74	6.95±3.17	0.13±0.15	0.27±0.21	0.25±0.18
IW-strict, 20% audits	0.53±0.18	3.95±0.64	2.64±0.70	0.30±0.14	7.47±2.84	4.68±2.45	0.06±0.05	0.16±0.13	0.21±0.14
IW-consensus	1.15±0.28‡	4.16±0.81	3.86±0.98	0.43±0.23‡	9.13±4.21	7.25±3.79	0.13±0.15‡	0.21±0.19	0.25±0.17

(3.26%). Vanilla's lower false alarms come with 99.6–99.8% of attacks getting through. Compared with normal operation (the first row of each block), false alarms rise 2.4–18× with evasion and 3.4–22× without it; this is the price of keeping evasion low.

G. RADP and Adversarial Training

On IoTID20, MSAAD's RADP filter mostly catches the impossible values of raw attacks rather than the attacks themselves (Table VII, 5 seeds). It flags only 10.9–17.1% of repaired attacks, against 91.4–100.0% of raw ones, so end-to-end evasion falls only from 15.96 to 12.98%. On CICIoT2023 it catches 98.0–99.8% of repaired attacks but deletes 26.36% of genuine benign flows. Separately, perturbing every flow, as the base paper does, produces FPRs of 65.12 / 78.38% (IoTID20 / CICIoT2023, 3 seeds), which explains the precision collapse reported there.

Adversarial training (15 seeds) stops transfer evasion on CICIoT2023 (48.52 → 0.03%) but more than doubles the clean FPR (6.41 → 14.13%, $p_{\text{Holm}} \leq 0.003$). Label-only HopSkipJump on valid traffic (5 seeds) still evades 25.20 / 51.20% of attacks against the plain detector and 14.00 / 2.00% against the adversarially trained one (IoTID20 / CICIoT2023).

H. Detector Chain

Adding an online MLP stage after the ensemble cuts no-label evasion on CICIoT2023 from 48.52 to 1.93%, but raises the FPR from 6.41 to 10.41%. A host-graph GNN stage is never significantly better than a parameter-matched host-feature MLP after Holm correction. Spoofing 10 source identities raises evasion after the full chain from 12.47 to 18.89% on IoTID20 (campaign order) and from 1.31 to 17.81% on CIC-IDS2017. For DDoS from few attacker hosts, simple per-host aggregates match message passing.

VI. DISCUSSION AND LIMITATIONS

An online detector's resistance to evasion is a property of its label channel, not only of its model. MSAAD resists evasion because its evaluation assumes an instant, error-free feedback loop; without labels, evasion rises to 17–49%, and delayed labels let 9–28% through. Any security evaluation of an adaptive online defence should state how labels arrive and test the detector with late, missing and poisoned labels.

Verified-label learning trades a modest audit budget for resilience to poisoning. Directing audits at uncertain flows makes this trade cheaper under attack: it uses fewer audits and cuts false alarms 8–13×.

TABLE V

AUDIT ALLOCATION WITH A 10% BUDGET. ASR UNDER EVASION PLUS FULL POISONING ($q = 1$); FPR AND AUDITS USED UNDER ATTACK WITH TRUE LABELS; FPR AT 99.9% DETECTION AND ROC AUC UNDER ATTACK AND ON CLEAN TRAFFIC. RANDOM: EACH FLOW AUDITED WITH PROBABILITY α; UNCERT.: THE MOST UNCERTAIN FLOWS (EQ. (1)); MIXED: HALF OF THE BUDGET EACH. AUDITS USED ARE COUNTS OUT OF 9,900 TEST FLOWS. ‡AS IN TABLE IV. –: NOT MEASURED.

		Default threshold			Under attack		Clean traffic	
Policy	Audits	ASR (q=1)	FPR attack	Audits used	FPR@99.9%	AUC	FPR@99.9%	AUC
IoTID20								
Vanilla	–	99.74±0.08	2.17±0.69	–	2.18±0.85	99.98±0.02	2.90±0.74	99.92±0.03
Strict	random	0.88±0.29	5.33±1.78	992±41	–	–	–	–
Strict	uncert.	0.28±0.18	2.45±0.73	911±26	–	–	–	–
IW-strict	random	1.15±0.28	3.52±0.74	992±41	3.92±1.35	99.95±0.02	3.46±0.95	99.84±0.06
IW-strict	uncert.	0.31±0.15	0.45±0.14	899±37	0.71±0.57	99.99±0.01	10.39±3.21	99.75±0.09
IW-strict	mixed	0.46±0.16	0.58±0.14	932±25	0.77±0.21	99.99±0.01	10.64±5.61	99.72±0.07
IW-consensus	random	1.15±0.28‡	3.86±0.98	992±41	9.40±6.32	99.94±0.04	3.53±1.02	99.82±0.07
IW-consensus	uncert.	0.31±0.15‡	0.48±0.17	885±43	1.81±3.17	99.98±0.02	11.18±3.90	99.75±0.07
CICIoT2023								
Vanilla	–	99.96±0.03	3.52±2.01	–	2.38±2.03	99.99±0.01	7.55±5.01	99.96±0.07
Strict	random	0.44±0.22	23.12±15.26	992±41	–	–	–	–
Strict	uncert.	0.45±0.41	3.78±2.75	902±43	–	–	–	–
IW-strict	random	0.43±0.23	6.95±3.17	992±41	5.87±5.26	99.98±0.01	8.65±8.15	99.93±0.10
IW-strict	uncert.	0.46±0.39	0.64±0.44	897±51	0.64±0.41	100.00±0.00	5.81±4.25	99.98±0.01
IW-strict	mixed	0.32±0.19	0.68±0.29	958±27	0.60±0.56	100.00±0.00	3.66±2.56	99.98±0.01
IW-consensus	random	0.43±0.23‡	7.25±3.79	992±41	10.91±9.04	99.96±0.03	8.69±6.38	99.95±0.07
IW-consensus	uncert.	0.46±0.39‡	0.84±0.52	912±41	1.58±1.29	99.99±0.00	6.11±5.88	99.98±0.02
CIC-IDS2017								
Vanilla	–	99.95±0.06	0.11±0.05	–	0.01±0.02	100.00±0.00	0.00±0.00	99.99±0.01
Strict	random	0.14±0.15	0.65±0.30	992±41	–	–	–	–
Strict	uncert.	0.13±0.12	0.11±0.07	818±95	–	–	–	–
IW-strict	random	0.13±0.15	0.25±0.18	992±41	0.02±0.03	100.00±0.00	0.01±0.03	99.99±0.01
IW-strict	uncert.	0.12±0.13	0.02±0.03	727±127	0.00±0.00	100.00±0.00	0.01±0.01	99.99±0.01
IW-strict	mixed	–	0.02±0.03	918±62	0.00±0.00	100.00±0.00	0.00±0.00	99.99±0.01
IW-consensus	random	0.13±0.15‡	0.25±0.17	992±41	0.02±0.03	100.00±0.00	0.01±0.03	99.99±0.01
IW-consensus	uncert.	0.12±0.13‡	0.02±0.03	732±107	0.00±0.00	100.00±0.00	0.00±0.01	99.99±0.01

Trusting only one label direction opens the door to the opposite attack. For example, strict updates stop poisoning but allow framing to succeed. While consensus updates combined with uncertainty audits limit both threats, a combined attack still pushes false alarms up to 2–26%.

If you deploy this system, we advise using IW-consensus with uncertainty audits backed by roughly a 10% budget. For smaller budgets, copy caps help by cutting false alarms about in half compared with a strict update strategy at a 5% budget. However, at a 2% audit budget, evasion climbs to 1.8–4.0%. Operators should expect more false alarms during an active poisoning and framing campaign.

Our findings have several limits. On the data side, we relied on laboratory captures with few attacker hosts and used flow features without packet replay; CIC-IDS2017 sits near a performance ceiling, and CICIoT2023 provides only about 330 benign test flows. Our attacker is a grey-box surrogate, not an adaptive adversary that targets the audit rule [40], and we treat a poisoning rate of $q = 1$ as a worst case. Finally, our seeds vary initializations, attacks and audit draws rather than the data sample itself, we use fixed copies instead of $1/p$ weights, the false-alarm increase on clean IoTID20 traffic is still unexplained, and MSAAD and RADP are our own reimplementations, with only 5 seeds for the RADP and HopSkipJump experiments.

All datasets are public, and the configuration, seeds and code for every run are available from the author.

VII. CONCLUSION AND FUTURE WORK

This work shows that an online detector cannot adapt safely without checking its incoming labels, and that verification should go where the model's confidence is lowest. Instant ground-truth labels keep evasion at or below 0.12%, but omitting or delaying them lets up to 49% of attack flows through. Even under simultaneous poisoning and framing, IW-consensus with uncertainty-directed audits keeps evasion below 1.7%, though operators must accept a higher false-alarm rate.

We see three items for future research. First, we need to model an adaptive attacker that targets the audit selection rule. Second, randomized audit selection with $1/p$ weights may remove the false-alarm increase we observed on clean traffic. Third, the system needs evaluation in real deployments with measured label delays and packet-level replay.

REFERENCES

[1] C. Kolias, G. Kambourakis, A. Stavrou, and J. Voas, "DDoS in the IoT: Mirai and other botnets," *Computer*, vol. 50, no. 7, pp. 80–84, 2017.

[2] Y. K. Beshah, S. L. Abebe, and H. M. Melaku, "Drift adaptive online DDoS attack detection framework for IoT system," *Electronics*, vol. 13, no. 6, p. 1004, 2024.

[3] L. Yang, D. M. Manias, and A. Shami, "PWPAE: An ensemble framework for concept drift adaptation in IoT data streams," in *Proc. IEEE Global Communications Conference (GLOBECOM)*, 2021, pp. 1–6.

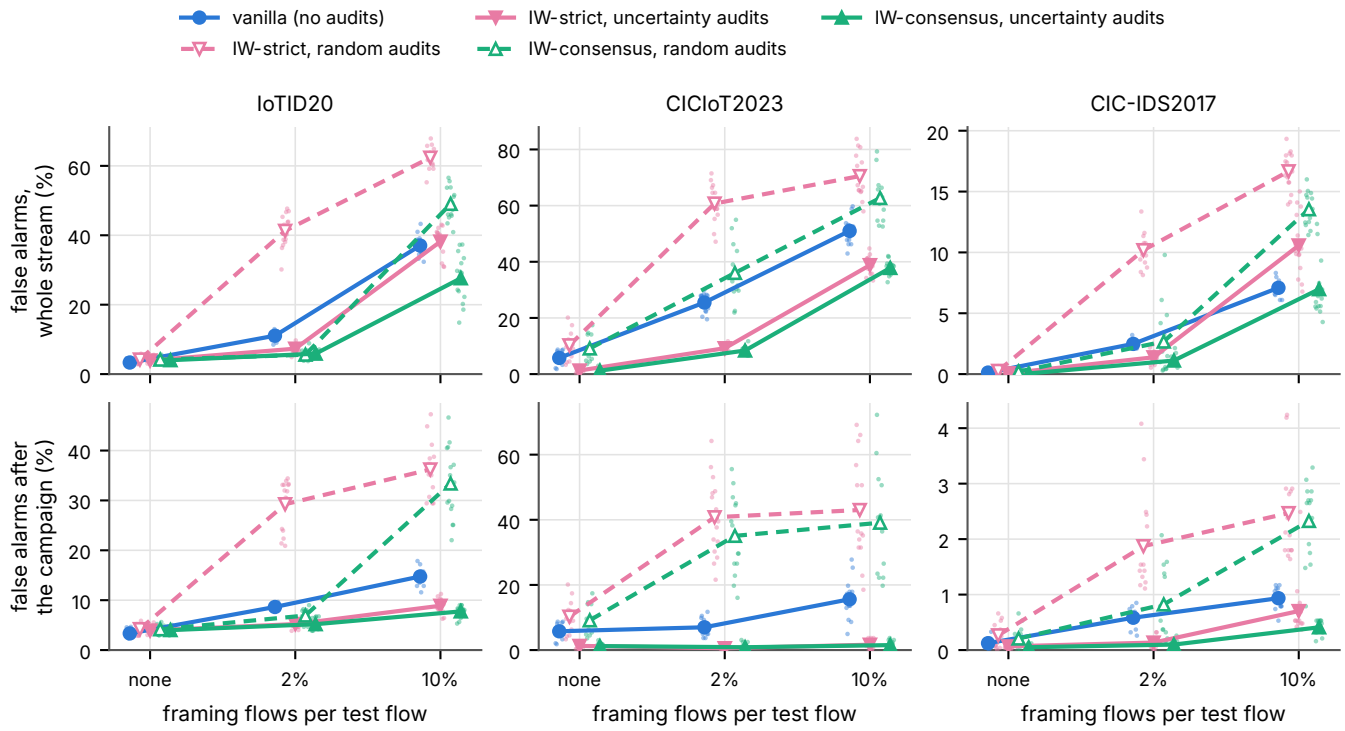


Fig. 4. False alarms on genuine benign flows over the whole stream (top) and after the framing campaign (bottom) as the framing rate r grows, with random and uncertainty-directed audits. Y-axis scales differ by dataset. Each dot is one seed.

[4] Y. K. Beshah, S. L. Abebe, and H. M. Melaku, "Multi-stage adversarial defense for online DDoS attack detection system in IoT," *IEEE Access*, vol. 13, pp. 72 657–72 673, 2025.

[5] I. Ullah and Q. H. Mahmoud, "A scheme for generating a dataset for anomalous activity detection in IoT networks," in *Advances in Artificial Intelligence (Canadian AI 2020)*, ser. LNCS, vol. 12109. Springer, 2020, pp. 508–520.

[6] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, and A. A. Ghorbani, "CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment," *Sensors*, vol. 23, no. 13, p. 5941, 2023.

[7] G. Engelen, V. Rimmer, and W. Joosen, "Troubleshooting an intrusion detection dataset: the CICIDS2017 case study," in *IEEE Security and Privacy Workshops (SPW)*, 2021, pp. 7–12.

[8] H. M. Gomes, A. Bifet, J. Read *et al.*, "Adaptive random forests for evolving data stream classification," *Machine Learning*, vol. 106, pp. 1469–1495, 2017.

[9] H. M. Gomes, J. Read, and A. Bifet, "Streaming random patches for evolving data stream classification," in *Proc. IEEE International Conference on Data Mining (ICDM)*, 2019, pp. 240–249.

[10] A. Bifet and R. Gavaldà, "Learning from time-changing data with adaptive windowing," in *Proc. SIAM International Conference on Data Mining (SDM)*, 2007, pp. 443–448.

[11] J. Gama, P. Medas, G. Castillo, and P. Rodrigues, "Learning with drift detection," in *Advances in Artificial Intelligence (SBIA 2004), LNCS 3171*. Springer, 2004, pp. 286–295.

[12] J. Montiel, M. Halford, S. M. Mastelini *et al.*, "River: Machine learning for streaming data in Python," *Journal of Machine Learning Research*, vol. 22, no. 110, pp. 1–8, 2021.

[13] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2018.

[14] L. Yang, W. Guo, Q. Hao, A. Ciptadi, A. Ahmadzadeh, X. Xing, and G. Wang, "CADE: Detecting and explaining concept drift samples for security applications," in *Proc. 30th USENIX Security Symposium*, 2021, pp. 2327–2344.

[15] M. Grzenda, H. M. Gomes, and A. Bifet, "Delayed labelling evaluation for data streams," *Data Mining and Knowledge Discovery*, vol. 34, no. 5, pp. 1237–1266, 2020.

[16] F. Ceschin, M. Botacin, A. Bifet, B. Pfahringer, L. S. Oliveira, H. M. Gomes, and A. Grégio, "Machine learning (in) security: A stream of problems," *Digital Threats: Research and Practice*, vol. 5, no. 1, pp. 9:1–9:32, 2024.

[17] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," in *Proc. International Conference on Learning Representations (ICLR)*, 2015.

[18] A. Kurakin, I. Goodfellow, and S. Bengio, "Adversarial examples in the physical world," in *ICLR Workshop Track*, 2017, arXiv:1607.02533.

[19] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards deep learning models resistant to adversarial attacks," in *Proc. International Conference on Learning Representations (ICLR)*, 2018.

[20] N. Carlini and D. Wagner, "Towards evaluating the robustness of neural networks," in *Proc. IEEE Symposium on Security and Privacy (S&P)*, 2017, pp. 39–57.

[21] J. Chen, M. I. Jordan, and M. J. Wainwright, "HopSkipJumpAttack: A query-efficient decision-based attack," in *Proc. IEEE Symposium on Security and Privacy (S&P)*, 2020, pp. 1277–1294.

[22] F. Pierazzi, F. Pendlebury, J. Cortellazzi, and L. Cavallaro, "Intriguing properties of adversarial ML attacks in the problem space," in *Proc. IEEE Symposium on Security and Privacy (S&P)*, 2020, pp. 1332–1349.

[23] G. Apruzzese, M. Andreolini, L. Ferretti, M. Marchetti, and M. Colajanni, "Modeling realistic adversarial attacks against network intrusion detection systems," *Digital Threats: Research and Practice*, vol. 3, no. 3, pp. 31:1–31:19, 2022.

[24] D. Arp, E. Quiring, F. Pendlebury *et al.*, "Dos and Don'ts of machine learning in computer security," in *Proc. 31st USENIX Security Symposium*, 2022, pp. 3971–3988.

[25] B. Biggio, B. Nelson, and P. Laskov, "Poisoning attacks against support vector machines," in *Proc. International Conference on Machine Learning (ICML)*, 2012.

[26] X. Zhang, X. Zhu, and L. Lessard, "Online data poisoning attacks," in *Proc. 2nd Conference on Learning for Dynamics and Control (L4DC)*, ser. PMLR, vol. 120, 2020, pp. 201–210.

TABLE VI

THE COMBINED ATTACK, WITH AND WITHOUT THE EVASION STEP. ASR AND FPR, %, MEAN ± STD OVER 15 SEEDS. THE FIRST ROW OF EACH BLOCK IS THE UNDEFENDED DETECTOR WITH TRUE LABELS AND NO POISONING OR FRAMING, FOR REFERENCE. –: NOT RUN.

		IoTID20		CICIoT2023		CIC-IDS2017	
Policy	Audits	ASR	FPR	ASR	FPR	ASR	FPR
Evasion (repaired PGD) + full poisoning + 5% framing							
Vanilla, true labels	–	0.10±0.03	2.17±0.69	0.08±0.03	3.52±2.01	0.02±0.01	0.11±0.05
Vanilla	–	99.74±0.08	8.75±1.67	99.93±0.04	29.83±3.98	99.95±0.06	3.51±0.31
Strict	random	0.23±0.09	71.63±7.75	0.16±0.17	76.28±8.44	–	–
Strict	uncert.	0.13±0.04	40.86±4.01	0.19±0.24	36.50±3.40	0.08±0.07	10.96±7.61
IW-strict	random	1.07±0.52	43.02±3.98	0.22±0.20	59.18±7.50	–	–
IW-strict	uncert.	0.58±0.19	17.21±4.94	0.21±0.24	17.94±2.70	0.12±0.10	3.51±1.10
IW-consensus	random	1.81±0.65	22.59±5.34	0.35±0.20	50.35±6.74	–	–
IW-consensus	uncert.	0.65±0.24	5.23±2.36	0.35±0.30	13.75±3.46	0.15±0.13	1.95±0.57
Full poisoning + 5% framing, no evasion							
Vanilla, true labels	–	0.05±0.03	3.33±0.63	0.10±0.04	5.77±2.24	0.05±0.01	0.12±0.08
Vanilla	–	99.61±0.05	7.16±1.26	99.72±0.04	25.01±3.70	99.84±0.04	3.26±0.34
Strict	random	0.19±0.18	73.72±4.48	0.02±0.03	82.32±6.91	–	–
Strict	uncert.	0.17±0.08	47.47±4.41	0.07±0.03	44.64±5.01	0.07±0.03	9.39±2.10
IW-strict	random	2.12±0.67	46.53±2.92	0.20±0.11	62.94±6.61	–	–
IW-strict	uncert.	1.36±0.33	23.85±5.62	0.23±0.09	27.91±2.36	0.12±0.05	3.26±0.72
IW-consensus	random	2.34±0.54	22.05±4.67	0.25±0.17	52.89±7.71	–	–
IW-consensus	uncert.	1.67±0.51	11.27±4.86	0.24±0.06	26.07±2.78	0.14±0.06	2.65±0.89

TABLE VII

BASELINES: RADP WITH THE DETECTOR FROZEN, PERTURBING EVERY FLOW, ADVERSARIAL TRAINING (AT) AND LABEL-ONLY HOPSKIPJUMP. VALUES IN %; RADP FLAG RANGES ARE MINIMUM–MAXIMUM OVER THE FOUR ATTACKS (FGSM, BIM, PGD, C&W); THE NUMBER OF SEEDS IS GIVEN PER BLOCK. –: NOT RUN.

Measure	IoTID20	CICIoT2023	CIC-IDS2017
RADP, our rebuild (5 seeds; ASR on repaired PGD)			
Repaired DDoS flagged	10.9–17.1	98.0–99.8	–
Raw DDoS flagged	91.4–100.0	98.6–100.0	–
Benign flows deleted	0.08±0.11	26.36±17.15	–
ASR, w/o→with RADP	15.96→12.98	56.23→0.51	–
Perturbing every flow, no labels (3 seeds)			
FPR	65.12±11.71	78.38±9.55	–
Adversarial training, no labels (15 seeds; repaired PGD)			
ASR, plain→AT	17.32→0.12	48.52→0.03	45.36→0.05
FPR, plain→AT	3.28→5.49	6.41→14.13	0.21→0.96
HopSkipJump, valid traffic (5 seeds)			
ASR, plain / AT	25.20 / 14.00	51.20 / 2.00	–

[27] G. Severi, S. Boboila, A. Oprea, J. Holodnak, K. Kratkiewicz, and J. Matterer, "Poisoning network flow classifiers," in *Proc. Annual Computer Security Applications Conference (ACSAC)*, 2023.

[28] J. Newsome, B. Karp, and D. Song, "Paragraph: Thwarting signature learning by training maliciously," in *Recent Advances in Intrusion Detection (RAID), LNCS 4219*. Springer, 2006, pp. 81–105.

[29] S. P. Chung and A. K. Mok, "Allergy attack against automatic signature generation," in *Recent Advances in Intrusion Detection (RAID), LNCS 4219*. Springer, 2006, pp. 61–80.

[30] D. Hendrycks, M. Mazeika, D. Wilson, and K. Gimpel, "Using trusted data to train deep networks on labels corrupted by severe noise," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 31, 2018.

[31] B. Settles, "Active learning literature survey," University of Wisconsin–Madison, Computer Sciences Technical Report 1648, 2009.

[32] A. Beygelzimer, S. Dasgupta, and J. Langford, "Importance weighted active learning," in *Proc. International Conference on Machine Learning (ICML)*, 2009, pp. 49–56.

[33] J. Lin, R. Luley, and K. Xiong, "Active learning under malicious mislabeling and poisoning attacks," in *Proc. IEEE Global Communications Conference (GLOBECOM)*, 2021, pp. 1–6.

[34] F. Tramèr, N. Carlini, W. Brendel, and A. Madry, "On adaptive attacks to adversarial example defenses," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.

[35] A. Athalye, N. Carlini, and D. Wagner, "Obfuscated gradients give a false sense of security: Circumventing defenses to adversarial examples," in *Proc. International Conference on Machine Learning (ICML)*, 2018, pp. 274–283.

[36] N. Carlini, A. Athalye, N. Papernot *et al.*, "On evaluating adversarial robustness," *arXiv preprint arXiv:1902.06705*, 2019.

[37] W. W. Lo, S. Layeghy, M. Sarhan, M. Gallagher, and M. Portmann, "E-GraphSAGE: A graph neural network based intrusion detection system for IoT," in *Proc. IEEE/IFIP Network Operations and Management Symposium (NOMS)*, 2022.

[38] W. Cong, S. Zhang, J. Kang *et al.*, "Do we really need complicated model architectures for temporal networks?" in *Proc. International Conference on Learning Representations (ICLR)*, 2023.

[39] S. Bhatia, B. Hooi, M. Yoon, K. Shin, and C. Faloutsos, "MIDAS: Microcluster-based detector of anomalies in edge streams," in *Proc. AAAI Conference on Artificial Intelligence*, 2020, pp. 3242–3249.

[40] Y. Zhi, L. Wang, X. Xie, C. Shen, Q. Hu, and X. Guan, "Selection-based vulnerabilities: Clean-label backdoor attacks in active learning," arXiv preprint arXiv:2508.05681, 2025.

[41] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Information Systems Security and Privacy (ICISSP)*, 2018, pp. 108–116.

[42] O. Chapelle and L. Li, "An empirical evaluation of Thompson sampling," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 24, 2011.

[43] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.

[44] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.